Ders 19 Çalışma Özeti

Ders 19 Çalışma Özeti

Genel Konular

- Dersin tarihi 29 Kasım 2021 Pazartesi. Ders çevrim içi (Zoom) yapıldı.
  - Slaytlar Silberschatz, Galvin ve Gagne, *Operating System Concepts*, 9. baskı, Bölüm 5 (Process Synchronization).
  - Asıl anlatım yaklaşık 00:06'da başlar. Ders 00:55-01:05 arasında ara verir.
- Vize notları ile ilgili duyuru
  - Notlar sisteme girildi ama henüz ilan edilmedi. Hoca bir kez daha kontrol edip ilan edecek.
  - Yüksek lisans öğrencileri notlarını e-postayla soracak.
- Senkronizasyona neden ihtiyaç var
  - Çoklu programlama (multiprogramming), süreçler arası iletişim (IPC) ve iş parçacıkları (thread) ortak veri paylaşır.
  - Ortak veriye eşzamanlı yazma verinin tutarsız (inconsistent) olmasına yol açabilir.
- Üretici-tüketici (producer-consumer) problemi ve `counter` örneği
  - `counter++` ve `counter--` makine düzeyinde üç adımdır: belleği yazmaca oku, artır/azalt, belleğe geri yaz.
  - Bu adımların arasında kesme olursa `counter` yanlış değer alır.
- Yarış durumu (race condition) ve kritik bölge (critical section)
  - Giriş bölümü (entry section), kritik bölge, çıkış bölümü (exit section), kalan bölüm (remainder section).
- Kritik bölge çözümünün üç şartı
  - Karşılıklı dışlama (mutual exclusion), ilerleme (progress), sınırlı bekleme (bounded waiting).
- Kesmeleri kapatma (interrupt disable) çözümü ve sakıncaları.
- Sırayla geçiş (yalnız `turn` kullanan) çözüm ve Peterson çözümü.
- Donanım desteği
  - Atomik komutlar: `test_and_set`, `compare_and_swap`.
- Mutex kilidi (mutex lock), spinlock ve meşgul bekleme (busy waiting).
- Semafor (semaphore)
  - `wait` ve `signal` işlemleri; ikili (binary) ve sayan (counting) semafor.
  - Semaforla sıralama (S_1, S_2'den önce çalışsın).
  - Bekleme kuyruğu ile meşgul beklemesiz semafor gerçeklemesi.
- Kilitlenme (deadlock) ve açlık (starvation)
  - İki semaforla (S ve Q) kilitlenme örneği.
  - Öncelik tersinmesi (priority inversion) ve öncelik kalıtımı (priority inheritance) kısaca.

Hocanın Özellikle Vurguladığı Kısımlar

- Kesme yalnız program satırları arasında olmaz.
  - Yüksek seviyeli tek bir satır birden fazla makine komutudur. Kesme bu komutların arasında da gelebilir.
- Yalnız okunan ortak veri sorun yaratmaz.
  - Okuma işlemi aynı sonucu verir, yani eşgüçlüdür (idempotent). Sorun, iki tarafın da yazdığı değişkendedir.
- İşletim sisteminde tek bir kritik bölge yoktur.

  - Kritik bölge, birbirleriyle veri paylaşan her süreç grubu için ayrı ayrı ortaya çıkar.
- Kesmeleri kapatmak "başı ağrıyanın başını kesmek" gibidir.
  - Mikrodenetleyicide kabul edilebilir: tek kullanıcı vardır, işletim sistemi yoktur.
  - Çok kullanıcılı, çok çekirdekli modern işletim sisteminde kabul edilemez.
- Peterson çözümü yalnız iki süreç için çalışır.
  - Süreç sayısı artınca kontroller arasında kesme olabilir ve çözüm bozulur. Bu yüzden donanım desteği gerekir.
- Donanım çözümleri uygulama geliştiriciye doğrudan açılmaz.
  - İşletim sistemi bunları mutex ve semafor gibi API'lerle sunar.
- Spinlock işlemci zamanını boşa harcar.
  - Uyutmak (sleep) çözüm değildir. Yalnız meşgul bekleme süresini azaltır, kaynağın ne zaman boşalacağını bilemezsiniz.
  - Doğru yaklaşım süreci bekleme kuyruğuna (wait queue) almak, kilit bırakılınca hazır kuyruğuna (ready queue) geri koymaktır.
- Çok işlemcili sistemde, kilit kısa süre tutulacaksa spinlock tercih edilir.
  - Bağlam değiştirmenin (context switch) maliyeti kısa beklemeden daha yüksektir.
- İşletim sistemi araçları sağlar; onları doğru kullanmak geliştiricinin sorumluluğudur.
  - Semaforu yanlış sırada kullanmak kilitlenmeye yol açar.
  - Tek başına doğru çalışan kod çok süreçli ortamda hatalı çalışabilir. Bu hataları bulmak (debugging) zordur.

Kısa Tekrar Notları

- Yarış durumu: Sonucun, süreçlerin ortak veriye hangi sırayla eriştiğine bağlı olması.
- Kritik bölge: Ortak veriyi değiştiren kod parçası. Aynı anda içinde yalnız bir süreç olmalı.
- Çözüm şartları:
  - Karşılıklı dışlama: Bir süreç kritik bölgedeyse diğerleri giremez.
  - İlerleme: Kritik bölge boşsa ve girmek isteyen varsa, giriş kararı süresiz ertelenemez.
  - Sınırlı bekleme: Bir süreç istek yaptıktan sonra diğerlerinin kaç kez girebileceğinin bir üst sınırı vardır.
- Peterson çözümü: Ortak `flag[2]` ve `turn` değişkenleri. Süreç `flag[i] = true` yapar, sırayı karşı tarafa verir (`turn = j`), sonra bekler.
- Atomik komut: Ortasında kesme olamayan komut (`test_and_set`, `compare_and_swap`; x86'da `XCHG`, `CMPXCHG`).
- Mutex: `acquire()` ve `release()`. Meşgul beklemeyle gerçeklenirse spinlock olur.
- Semafor: Tam sayı değişken. `wait()` değeri azaltır, gerekirse süreci bekletir. `signal()` değeri artırır, gerekirse bekleyen bir süreci uyandırır.
  - İkili semafor: 0 veya 1, kullanım olarak mutex'e benzer.
  - Sayan semafor: Birden fazla eş kaynağı korur.
- Kilitlenme: İki ya da daha fazla süreç, yalnız birbirinin yapabileceği bir olayı bekler.
- Açlık: Bir süreç kaynağa süresiz olarak erişemez.

Detaylı Açıklamalar

`counter` örneğinde yarış durumu

Üretici `counter++`, tüketici `counter--` çalıştırır. Makine düzeyinde:

```
/* counter++ */              /* counter-- */
register1 = counter;         register2 = counter;
register1 = register1 + 1;   register2 = register2 - 1;
counter = register1;         counter = register2;
```

Başlangıçta `counter = 5` olsun. Üretici `register1 = 6` yaptıktan sonra, değeri belleğe yazmadan kesilsin. Tüketici `counter` değerini hâlâ 5 olarak okur ve 4 hesaplar. Hangi süreç son yazarsa `counter` 4 ya da 6 olur. Doğru değer 5'tir: bir artırma ve bir azaltma yapıldı.

Sırayla geçiş çözümü

```
while (true) {
    while (turn == j) ;   /* bekle */
```

```
    /* kritik bölge */
    turn = j;
    /* kalan bölüm */
}
```

Her süreç işi bitince sırayı karşı tarafa verir. Hoca bunu "nazik" bir yaklaşım olarak anlattı ve ağlardaki Aloha protokolüne benzetti.

Peterson çözümü (iki süreç)

```
/* ortak: int turn; bool flag[2]; */
while (true) {
    flag[i] = true;
    turn = j;
    while (flag[j] && turn == j) ;   /* bekle */
    /* kritik bölge */
    flag[i] = false;
    /* kalan bölüm */
}
```

- `flag[i]`: P_i kritik bölgeye girmeye hazır.
- `turn`: Sıranın kimde olduğu. Her süreç sırayı karşı tarafa verir.
- Varsayım: `load` ve `store` komutları atomiktir.

`test_and_set` ve `compare_and_swap`

```
bool test_and_set(bool *target) {   /* atomik */
    bool rv = *target;
    *target = true;
    return rv;
}

/* kullanım: lock başlangıçta false */
while (test_and_set(&lock)) ;   /* bekle */
/* kritik bölge */
lock = false;

int compare_and_swap(int *value, int expected, int new_value) {   /* atomik */
    int temp = *value;
    if (*value == expected)
        *value = new_value;
    return temp;
}
```

`test_and_set` eski değeri döndürür. Dönen değer `false` ise kilit boştu ve artık bu sürecindir. `compare_and_swap` değeri yalnız beklenen değere eşitse değiştirir. Bu işlemlere sınırlı bekleme şartını da eklemek kodu karmaşıklaştırır.

Spinlock mu, bekleme kuyruğu mu?

- Spinlock: Süreç kilidi bir döngüde sürekli dener (meşgul bekleme). Hoca bunu çizgi filmlerdeki "geldik mi, geldik mi?" diye soran çocuğa benzetti.
- Bekleme kuyruğu: Kilidi alamayan süreç bekleme durumuna geçer. Kilit bırakılınca hazır kuyruğuna alınır.
  - Tek işlemcide bu yaklaşım daha iyidir: işlemci boşa dönmez.
  - Çok işlemcide ve kısa kritik bölgede spinlock daha verimli olabilir: bağlam değiştirme maliyeti yoktur.

Semaforla sıralama

S_1 deyimi (P_1'de) S_2 deyiminden (P_2'de) önce çalışsın. `synch` semaforu 0 ile başlatılır.

```
/* P1 */            /* P2 */
S1;                 wait(synch);
signal(synch);      S2;
```

P_2 `wait(synch)` çağrısında bloklanır. P_1, S_1'i bitirip `signal(synch)` yapınca P_2 devam eder.

Semaforla kilitlenme örneği

S ve Q semaforları 1 ile başlatılır.

```
/* P0 */        /* P1 */
wait(S);        wait(Q);
wait(Q);        wait(S);
...             ...
signal(S);      signal(Q);
signal(Q);      signal(S);
```

P_0 `wait(S)` yapar ve kesilir. P_1 `wait(Q)` yapar. Şimdi P_0 Q'yu, P_1 de S'yi bekler. İkisi de ilerleyemez.

Ders sonrası notlar

- Hocanın 00:41'de "Peterson çözümüne benzer" diye gösterdiği, yalnız `turn` kullanan çözüm aslında sırayla geçiştir (strict alternation). Karşılıklı dışlamayı sağlar ama ilerleme şartını sağlamaz. Örnek: P_j hiç girmek istemese bile P_i iki kez üst üste giremez. Peterson çözümü bu sorunu `flag` dizisiyle giderir.
- Hocanın sorduğu "kesme ne zaman kontrol edilir?" sorusu derste cevapsız kaldı. Cevap: İşlemci kesme isteğini her komut döngüsünün sonunda, bir sonraki komutu getirmeden (fetch) önce kontrol eder. Bu yüzden kesme, `counter++` gibi bir satırı oluşturan makine komutlarının arasına girebilir.
- `test_and_set` anlatımında hoca bir an "true'ya set ettik, pardon false olarak kaldı" dedi. Doğrusu: `test_and_set` kilidi her durumda `true` yapar ve eski değeri döndürür. Eski değer `false` ise kilit alınmıştır, döngüden çıkılır.
- Hoca semaforda `signal` işleminin "bekleyen tüm süreçleri" hazır kuyruğuna aldığını söyledi. Kitaptaki tanımda `signal` yalnız bir süreci uyandırır (`wakeup(P)`). Bütün bekleyenleri uyandırmak ayrı bir işlemdir (ör. koşul değişkenlerinde `broadcast`).
- Starvation çözümü olarak anlatılan "bekleyen sürecin önceliğini zamanla artırmak" yaşlandırma (aging) diye adlandırılır.
- Hoca "priority inversion yaparsınız, inherit etmesini sağlarsınız" dedi. Öncelik tersinmesi (priority inversion) sorunun adıdır: düşük öncelikli süreç, yüksek öncelikli sürecin beklediği kilidi tutar. Çözüm öncelik kalıtımıdır (priority inheritance): kilidi tutan süreç, kilidi bırakana kadar bekleyen sürecin yüksek önceliğini geçici olarak alır.
- Hoca "kernel preemptive değilse kernel modunda kritik bölgeden bahsetmek mümkün değil" dedi. Bu tek işlemcide doğrudur. Çok işlemcili sistemde iki çekirdek aynı anda kernel kodu çalıştırabilir; preemptive olmayan kernelde de kilit gerekir.
- Peterson çözümü modern mimarilerde ek önlem olmadan güvenli değildir. İşlemci ve derleyici bellek işlemlerinin sırasını değiştirebilir. Doğru çalışması için bellek bariyeri (memory barrier) gerekir. Kitap da bunu belirtir.
- Hoca PowerPC'de `compare_and_swap` biçiminde komutlar olduğunu söyledi. PowerPC atomik işlemleri `lwarx`/`stwcx.` (load-linked/store-conditional) komut çiftiyle yapar. Doğrudan `compare_and_swap` biçimindeki komut x86'daki `CMPXCHG`'dir.
- Altyazıdaki "aydan potent" eşgüçlü (idempotent), "shape memory" paylaşımlı bellek (shared memory), "malta korunumuz" çok çekirdek (multi-core), "require" `acquire`, "weight" `wait` demektir.

Not: İsterseniz bu dersin altyazı (.srt) dosyasını NotebookLM gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.